CS3281 / CS5281

# I/O Devices

CS3281 / CS5281

Spring 2026

**Some lecture slides borrowed and adapted from Andrea Arpaci-Dusseau, Branden Ghena*

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

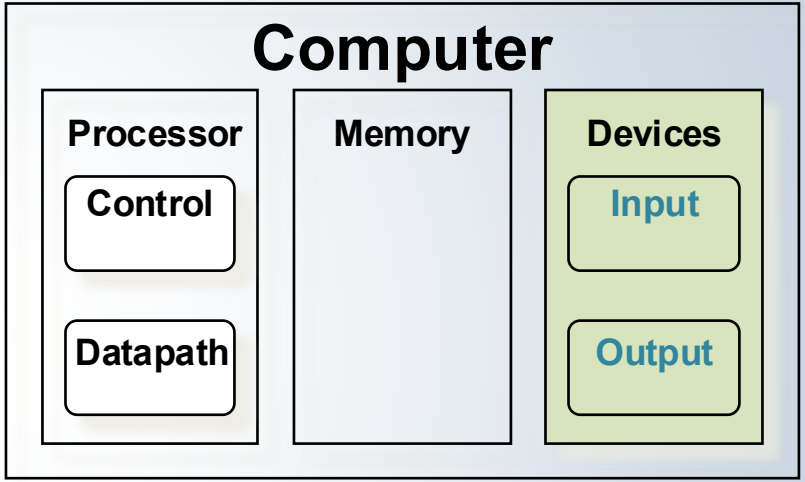
VANDERBILT UNIVERSITY

# Outline

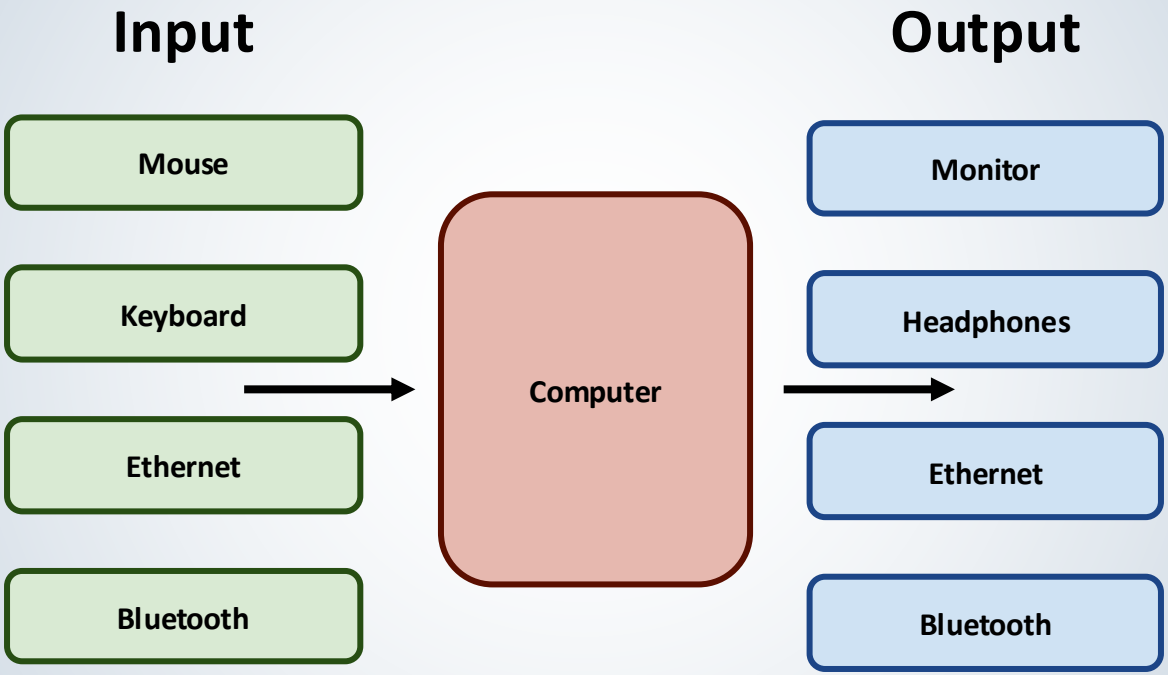
- **Overview of Device I/O**
- Connecting to devices
  - Buses on a computer
- Talking to devices

# Devices are the point of modern computers

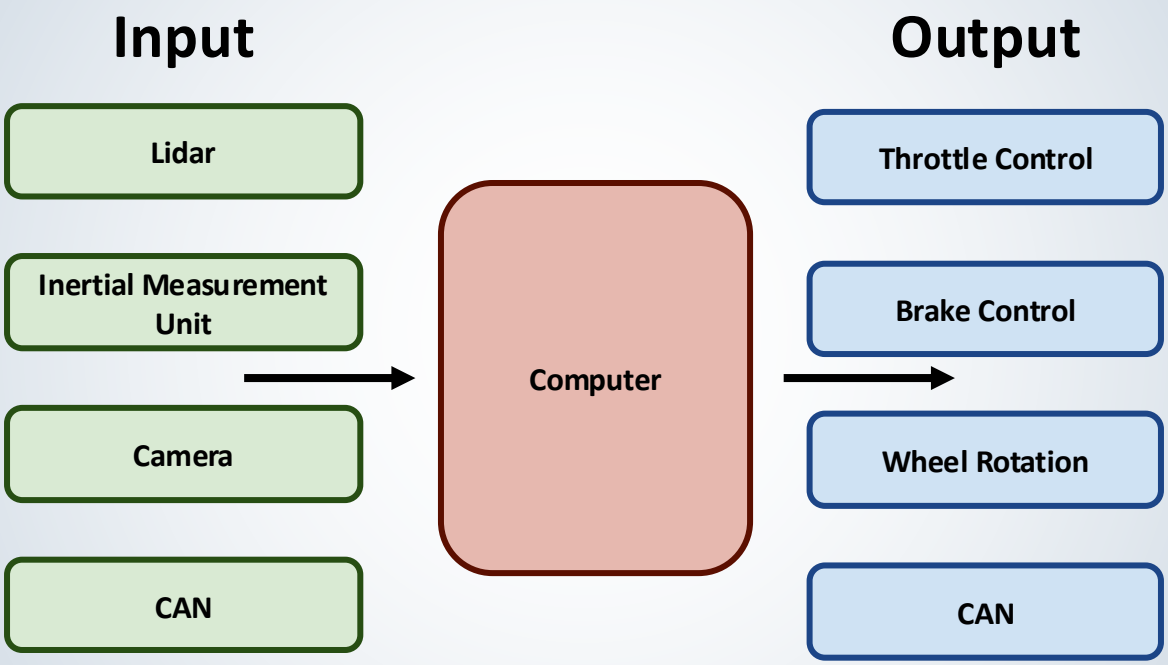
- Computation was sufficient for batch systems
  - Even then, tape or disk was the input and output mechanism
- But interactive systems need to receive input from users and output responses
  - Keyboard/mouse
  - Disk
  - Network
  - Graphics
  - Audio
  - Various USB devices



# Devices are core to useful general-purpose computing



# Devices are essential to cyber-physical systems too



# Device access rates vary by many orders of magnitude

- Rates in bit/sec
- System must be able to handle each of these
  - Sometimes needs low overhead
  - Sometimes needs to not wait around

| Device | Behavior | Partner | Data Rate (Kb/s) |
|---|---|---|---|
| Keyboard | Input | Human | 0.2 |
| Mouse | Input | Human | 0.4 |
| Microphone | Output | Human | 700.0 |
| Bluetooth | Input or Output | Machine | 20,000.0 |
| Hard disk drive | Storage | Machine | 100,000.0 |
| Wireless network | Input or Output | Machine | 300,000.0 |
| Solid state drive | Storage | Machine | 500,000.0 |
| Wired LAN network | Input or Output | Machine | 1,000,000.0 |
| Graphics display | Output | Human | 3,000,000.0 |

# Handling devices appropriately

- OS concerns
  - Communicating with devices needs to be fast and efficient
  - Devices are shared resources that need to be access controlled and shared
  - Devices are wildly variable and need some common interfaces for application software

# Recurring Theme

OS needs:

1. Virtualization
2. Concurrency
3. Persistence

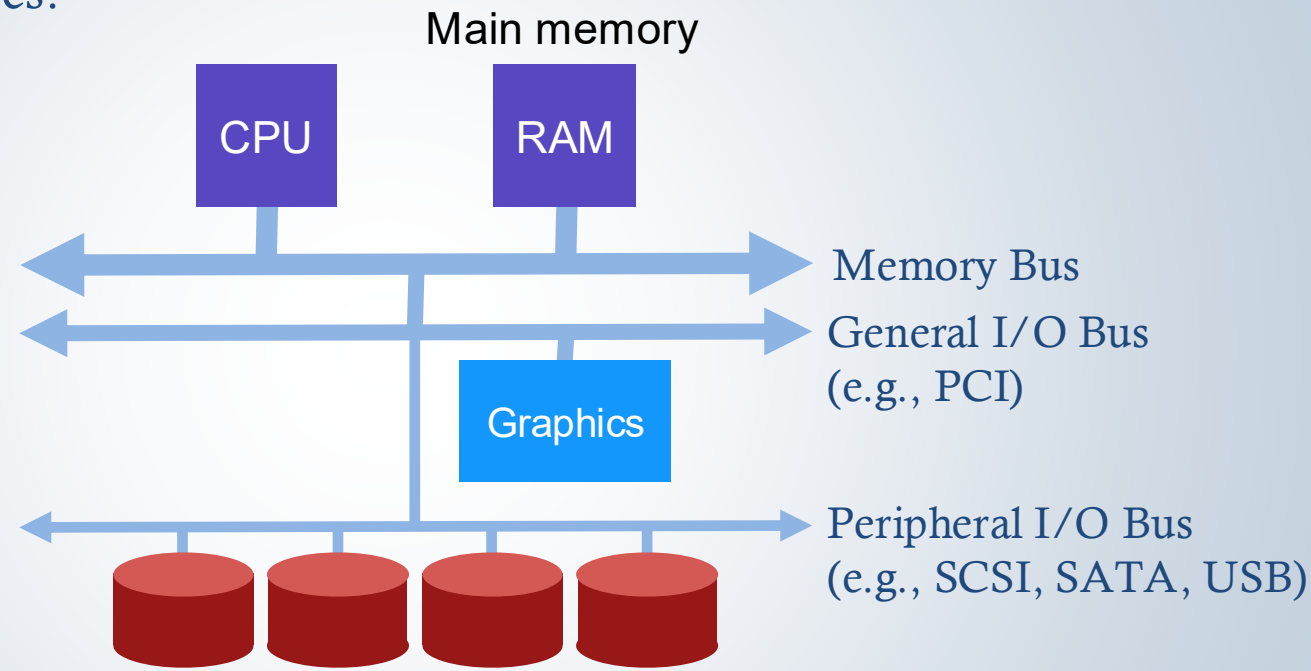
- I/O allows for *persistence*
  - RAM is *volatile*, i.e., contents are lost when the machine restarts

# Outline

- Overview of Device I/O
- **Connecting to devices**
  - **Buses on a computer**
- Talking to devices

# Hardware support for I/O

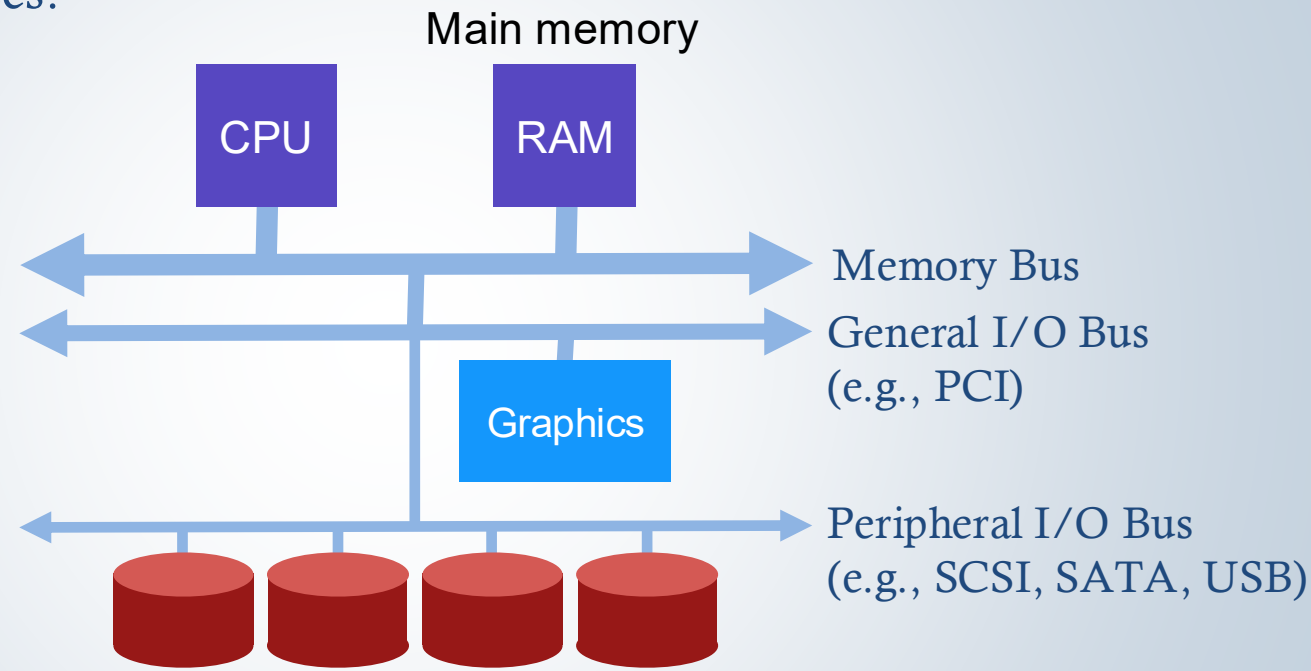
Why use hierarchical buses?



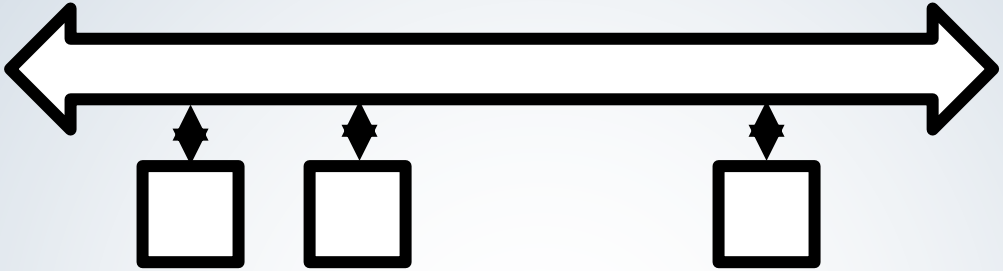
# Hardware support for I/O

## Why use hierarchical buses?

- Close to the CPU are very fast connections
- Farther from CPU are slower but more flexible protocols



# What is a bus anyways?

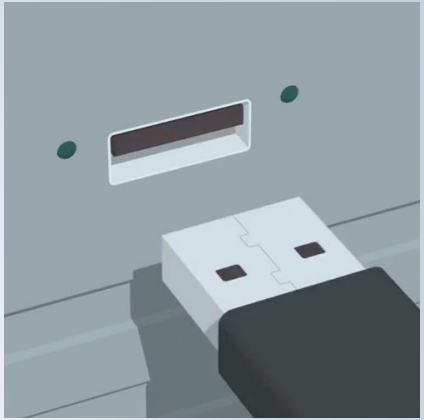


- Common set of wires for communication between two or more components
  - Lets one set of wires connect to N devices
- Standardized buses have a specific set of wires and protocol for communicating over them (DMI, PCI, SATA, USB)
  - Example wires: 64 address wires, 64 data wires, ~10 control wires
- Concerns
  - How many wires in the bus?
  - Single controller or multiple with arbitration?
  - Half-duplex (one direction of communication at a time) or full-duplex?

# Computer networks also run over buses

- Ethernet is a bus too!
- Network protocols specify how two computers communicate, very similarly to these buses
- Compared to networks, internal computer busses have:
  - Higher speed
  - Very high reliability
  - Accessed cooperatively (often with only one controller: the CPU)

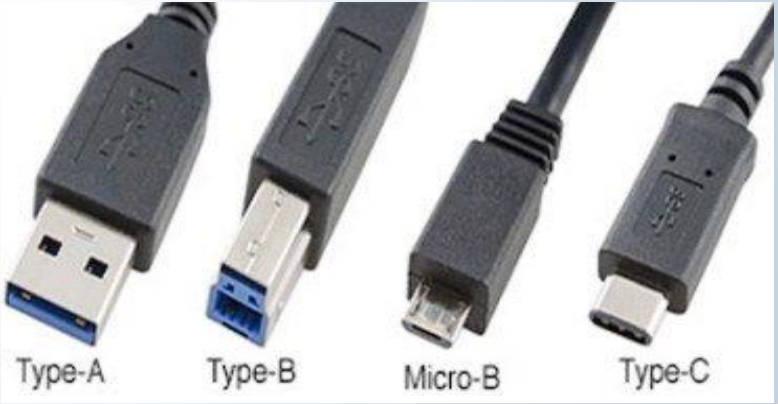
# USB – Universal Serial Bus



- "That one you are familiar with already"

- Connect and power external devices
- Comes in multiple form-factors and versions
  - USB 3.x and type C connectors are the fast ones

- Speed has increased greatly
  - 1.5 Mbps
  - 12 Mbps
  - 480 Mbps
  - 5 Gbps
  - 10 Gbps
  - 40 Gbps

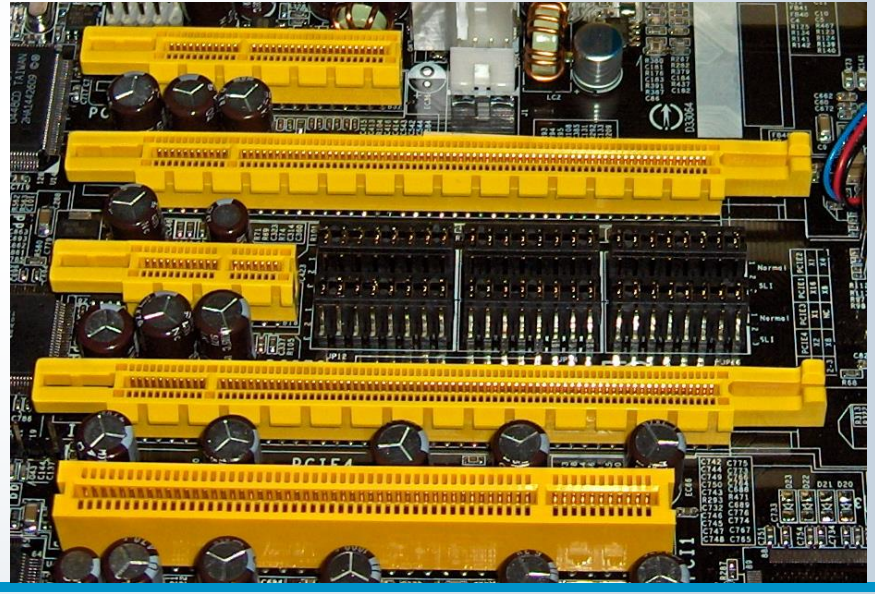


# SATA – Serial ATA – AT Attachment – “Advanced Technology”

- “That cable you plug an SSD/HDD in with”
- Long evolution as you might have guessed from the name
- 6 Gbps
  - Short connection length for high speed

# PCIe – Peripheral Component Interconnect Express

- "That slot you put a graphics card into"
  - Or WiFi card, or some SSDs
- Collection of point-to-point connections
  - Motherboard and CPU support N "lanes" in various configurations
- Different sizes for different number of bits in parallel
  - Order 10s Gbps



# Outline

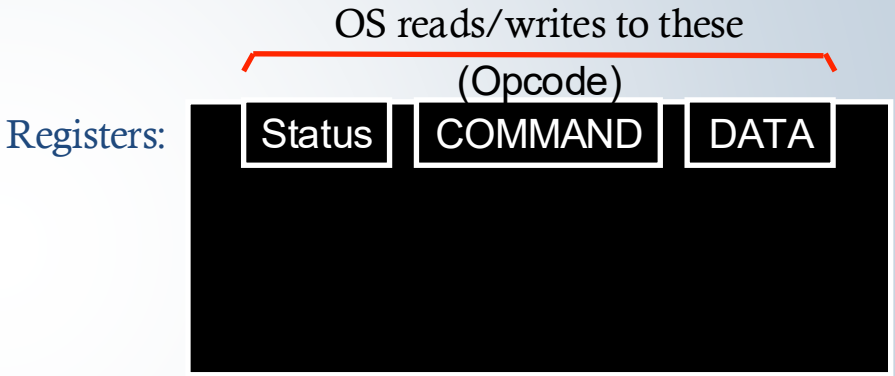
- Overview of Device I/O
- Connecting to devices
  - Buses on a computer
- **Talking to devices**

# Motivation

- Device controller is a circuit that operates (controls) the specific I/O devices
- To do the operation, the controller has registers and flags that are set and examined by the device *drivers*
- A device driver is a program that translates the user application *generic* I/O operations to interact with the device controller. The translated operations are used to control the specific device

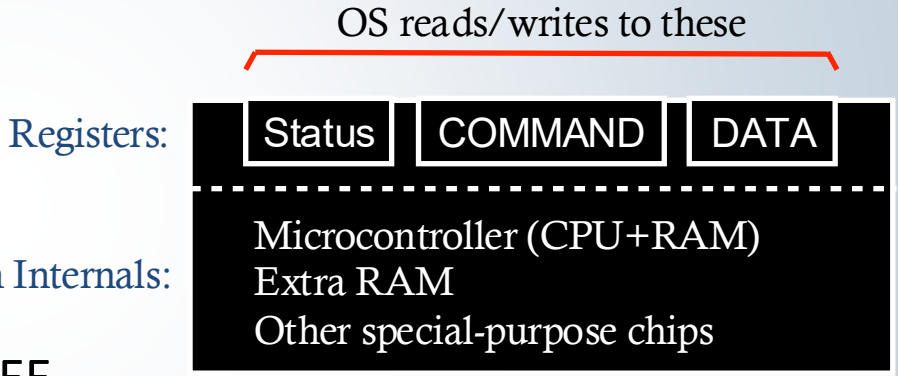
# Canonical Device Controller

- How do we read/write them?
  - Special assembly instructions
  - Treat like normal memory

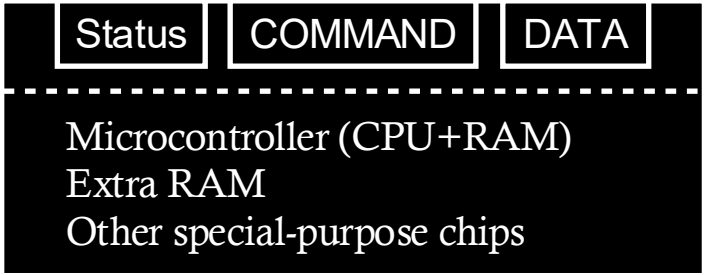


# Canonical Device Controller

- A device is really a miniature computer-within-the-computer
  - Has its own processing, memory, software
- We can mostly ignore that and deal with its interface
  - Called registers (actually are from EE perspective, but you can't use them)
  - Read/Write like they're data



# Example Write Protocol




```
while (STATUS == BUSY)
    ; // spin
Write data to DATA register
Write command to COMMAND register
while (STATUS == BUSY)
    ; // spin
```

Wait for device to be free

Wait for write to finish

This is called <u>polling</u> when the processor keeps "asking" what the hardware is doing by reading *status* flag.

CPU:

Disk:

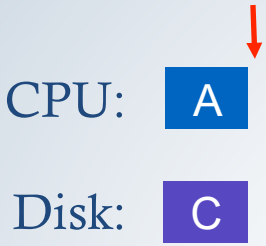
```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register      // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```

CPU: A

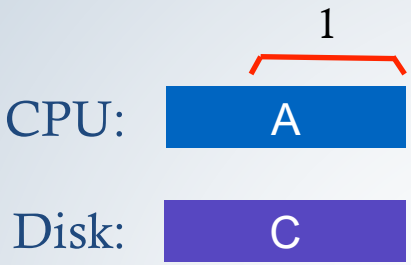
Disk: C

```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register      // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```

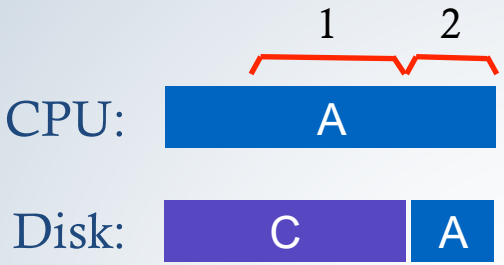
A wants to do I/O



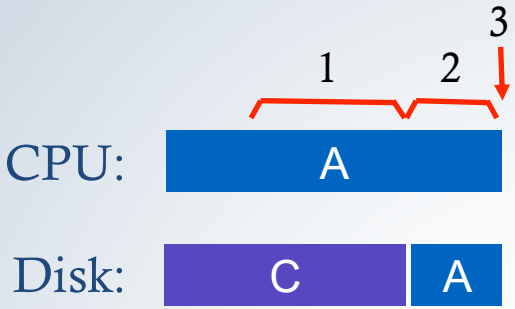

```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register      // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```



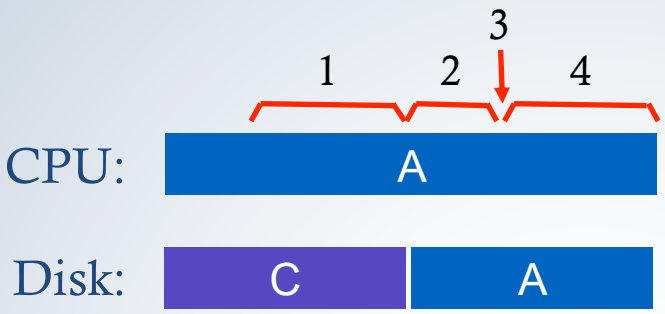

```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register      // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```



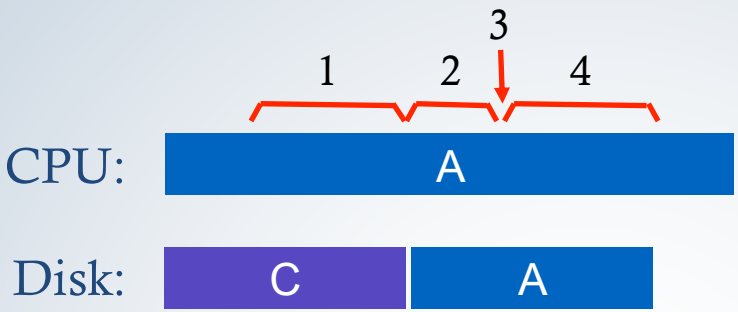

```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```




```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```

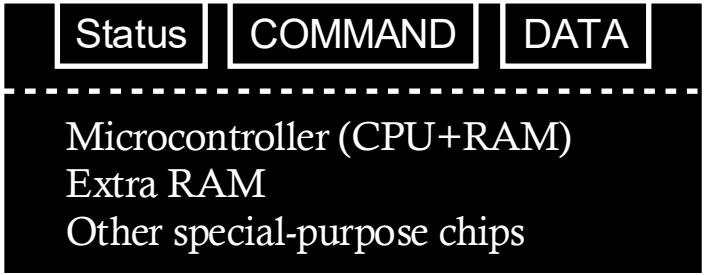



```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```



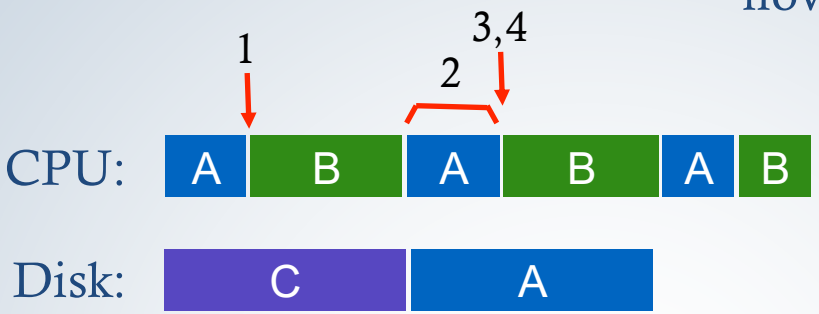

```
while (STATUS == BUSY)          // 1
    ;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    ;
```

# Example Read Protocol




```
while (STATUS == BUSY)
    ; // spin
Write command to COMMAND register
while (STATUS == BUSY)
    ; // spin
Read data from DATA register
```

# how to avoid spinning? interrupts!




A process blocks if a device is busy. When the device is not busy, controller issues an interrupt to CPU and unblocks the process

```
while (STATUS == BUSY)          // 1
    wait for interrupt;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
    wait for interrupt;
```

# Interrupts vs. Polling

Are interrupts ever worse than polling?

- Interrupts represent *predictable* overhead for each I/O operation. CPU time is not wasted on spinning when a device is slow
- Fast device or a single process running: Better to poll (spin) than take interrupt and context-switching overhead
- Device time unknown? Hybrid approach (spin then use interrupts)

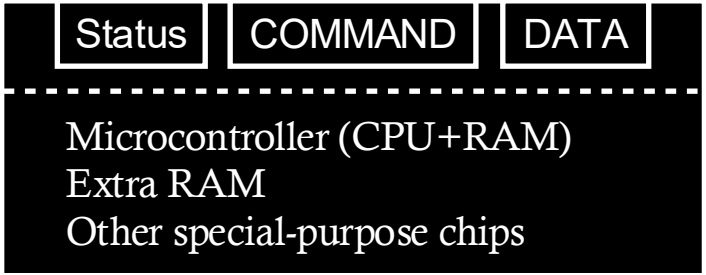
Flood of interrupts arrive for packet arrivals

- Can lead to livelock (always handling interrupts)
- Use polling for high load of arrivals and interrupts for low load of arrivals (hybrid)

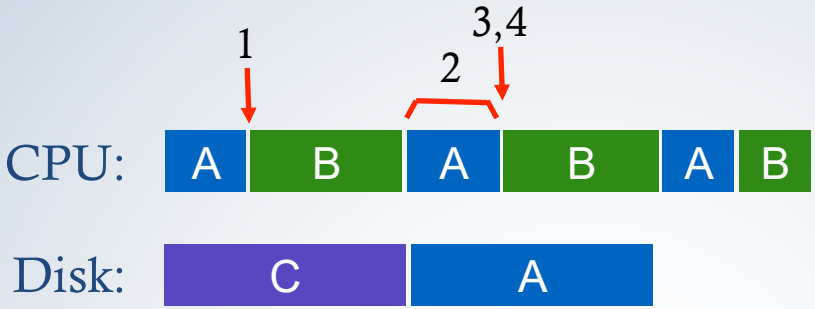
Other improvement

- Interrupt coalescing (hardware batches together several interrupts)

# Protocol Variants




- **Status checks**: polling *vs.* interrupts
- **Data**: programmed I/O (PIO) *vs.* direct memory access (DMA)
- **Control**: special instructions *vs.* memory-mapped I/O




```
while (STATUS == BUSY)          // 1
        wait for interrupt;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
        wait for interrupt;
```

what else can we optimize?

data transfer!

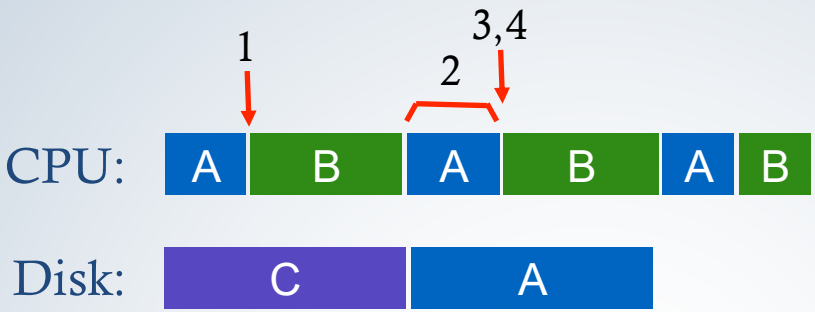
# Programmed I/O vs. Direct Memory Access

**PIO** (Programmed I/O):

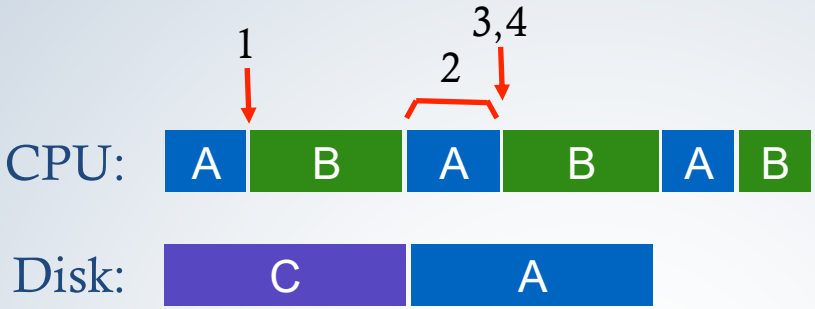
- CPU directly copies data between the device controller and main memory
- Efficient for a few bytes/words, but *scales terribly*

**DMA** (Direct Memory Access):

- CPU leaves data in memory and just initiates an I/O operation
- Device reads/writes data directly from/to memory using the DMA device without involving CPU
- CPU is free to serve the other processes
- Efficient for large data transfers



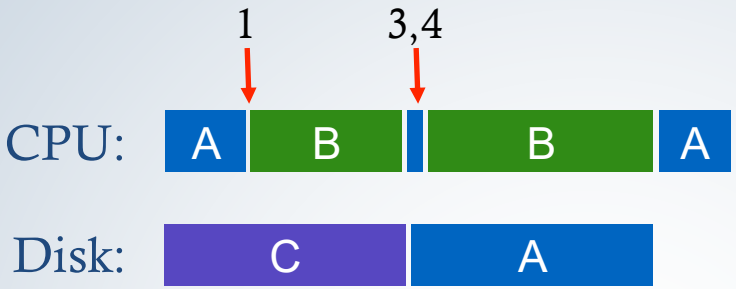

```
while (STATUS == BUSY)          // 1
        wait for interrupt;
Write data to DATA register     // 2
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
        wait for interrupt;
```




```
while (STATUS == BUSY)          // 1
        wait for interrupt;
```

~~Write data to DATA register  // 2~~

```
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
        wait for interrupt;
```



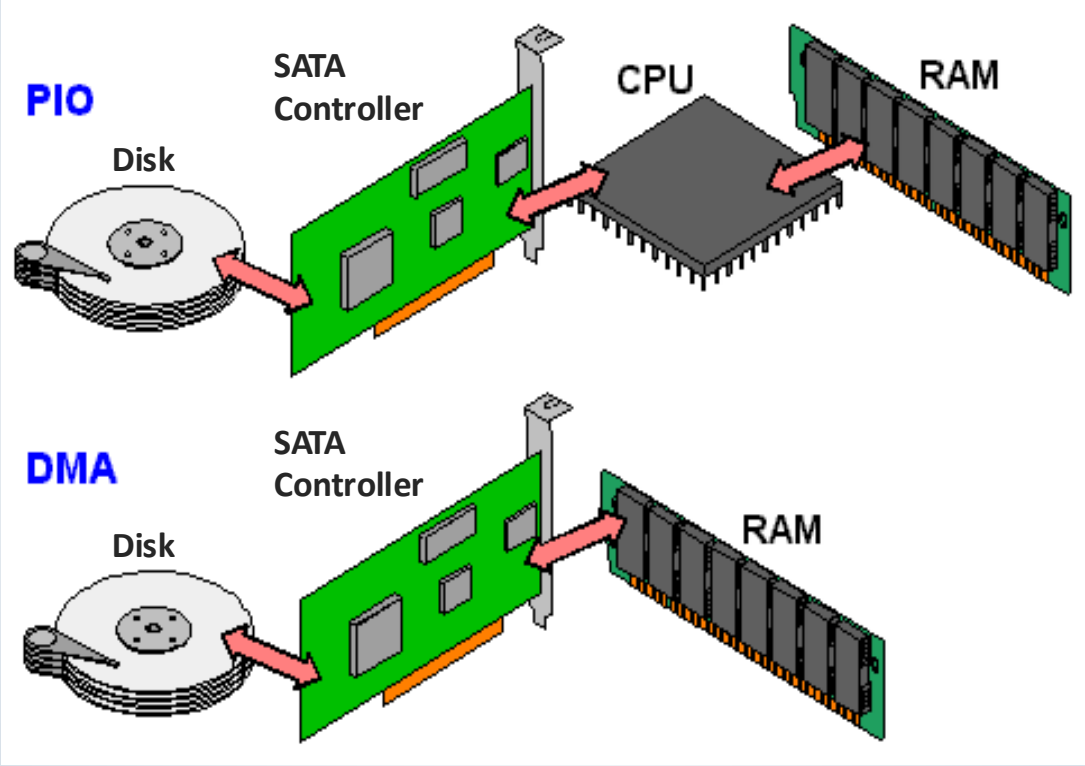

```
while (STATUS == BUSY)          // 1
        wait for interrupt;
```

~~Write data to DATA register  // 2~~

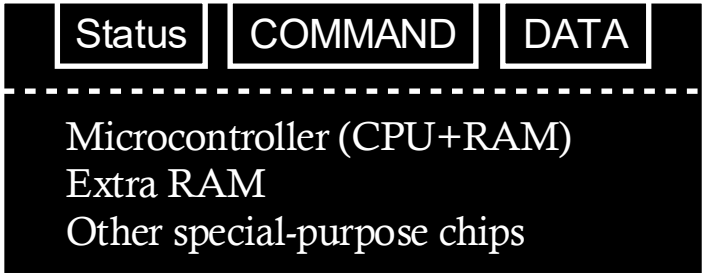
```
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
        wait for interrupt;
```

Device controller issues an interrupt to CPU to unblock A

# Programmed I/O versus Direct Memory Access



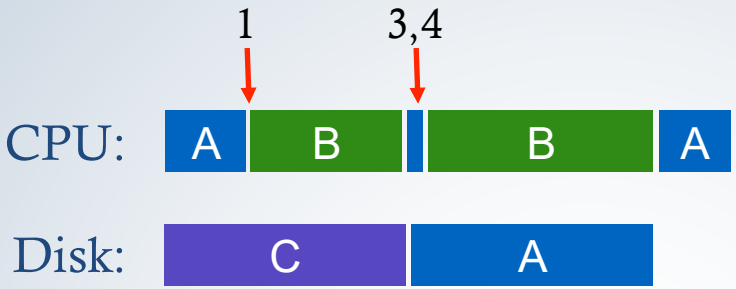
# Protocol Variants




**Status checks**: polling *vs.* interrupts

**Data**: PIO *vs.* DMA

**Control**: special instructions *vs.* memory-mapped I/O



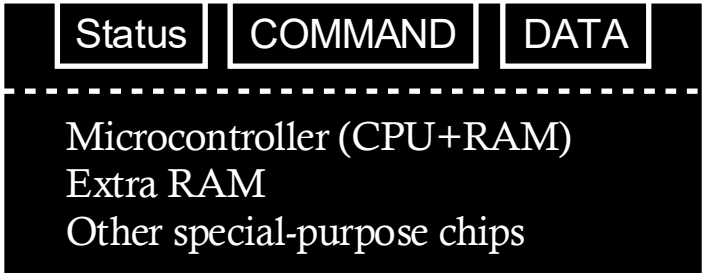

```
while (STATUS == BUSY)          // 1
        wait for interrupt;
```

~~Write data to DATA register  // 2~~

```
Write command to COMMAND register  // 3
while (STATUS == BUSY)          // 4
        wait for interrupt;
```

how does OS read and write registers?

# Protocol Variants




**Status checks**: polling *vs.* interrupts

**Data**: PIO *vs.* DMA

**Control**: special instructions *vs.* memory-mapped I/O

# Special Instructions vs. Memory-Mapped I/O

Special instructions

- each device has a port
- in/out instructions (x86) communicate with device

Memory-Mapped I/O

- H/W maps registers into the virtual address space
- loads/stores sent to device

# Port-Mapped I/O (PMIO): special assembly instructions

- x86 IN and OUT instructions
  - Privileged instructions (kernel mode only)
  - Two arguments: destination and data register

- Each device is mapped to some port address
  - IN and OUT instructions interact with interface

  - IN <PORT NUMBER>, <REGISTER>
  - OUT <REGISTER>, <PORT NUMBER>

# Example powered device: Real Time Clock

- Example: read current value from real-time clock

| Index | Contents | Range |
|---|---|---|
| 0x00 | Seconds | 0-59 |
| 0x02 | Minutes | 0-59 |
| 0x04 | Hours | 0-23 in 24-hour mode,<br>1-12 in 12-hour mode, highest bit set if PM |
| 0x06 | Weekday | 1-7, Sunday =1 |
| 0x07 | Day of Month | 1-31 |
| 0x08 | Month | 1-12 |
| 0x09 | Year | 0-99 |

```
// read seconds
mov $0, %al
out %al, $0x70
in $0x71, %al
```

| Index Register | Data Register |
|---|---|
| 0x70[1] | 0x71 |

Port Address

https://www.singlix.com/trdos/archive/pdf_archive/real-time-clock-nmi-enable-paper.pdf

# Example I/O port map

This isn't standardized, but these are some typical values.

https://wiki.osdev.org/Can_I_have_a_list_of_IO_Ports

| Port range | Summary |
|---|---|
| 0x0000-0x001F | The first legacy DMA controller, often used for transfers to floppies. |
| 0x0020-0x0021 | The first Programmable Interrupt Controller |
| 0x0022-0x0023 | Access to the Model-Specific Registers of Cyrix processors. |
| 0x0040-0x0047 | The PIT (Programmable Interval Timer) |
| 0x0060-0x0064 | The "8042" PS/2 Controller or its predecessors, dealing with keyboards and mice. |
| 0x0070-0x0071 | The CMOS and RTC registers |
| 0x0080-0x008F | The DMA (Page registers) |
| 0x0092 | The location of the fast A20 gate register |
| 0x00A0-0x00A1 | The second PIC |
| 0x00C0-0x00DF | The second DMA controller, often used for soundblasters |
| 0x00E9 | Home of the Port E9 Hack. Used on some emulators to directly send text to the hosts' console. |
| 0x0170-0x0177 | The secondary ATA harddisk controller. |
| 0x01F0-0x01F7 | The primary ATA harddisk controller. |
| 0x0278-0x027A | Parallel port |
| 0x02F8-0x02FF | Second serial port |
| 0x03B0-0x03DF | The range used for the IBM VGA, its direct predecessors, as well as any modern video card in legacy mode. |
| 0x03F0-0x03F7 | Floppy disk controller |
| 0x03F8-0x03FF | First serial port |

# Annoying parts of Port-Mapped I/O

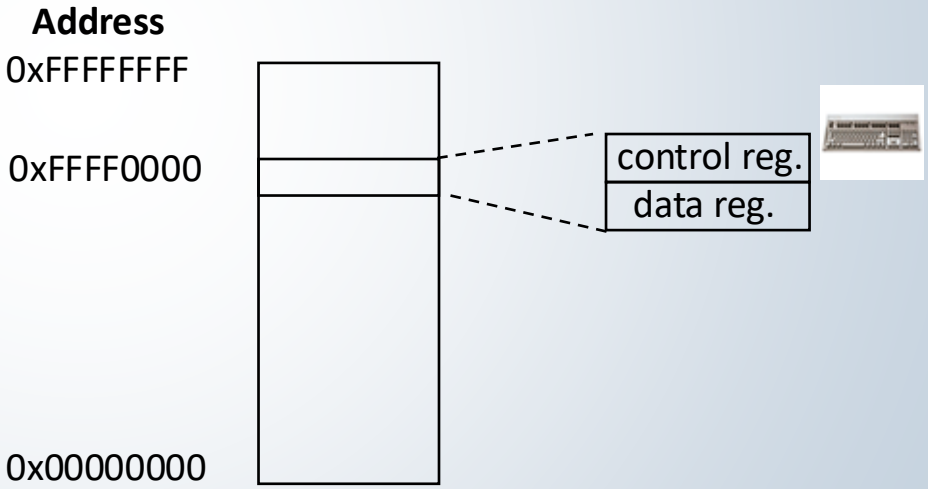
- Special assembly instructions are hard to write in C
  - Need some wrapper function that actually calls them
  - Not really that big of an issue, but a little weird

- Feels sort of like memory read/write, but isn't
  - Why not?
  - Can we just put the "port address space" somewhere in memory?
    - Could be a problem if we don't have enough memory
    - But today we have tons of extra physical address space laying around

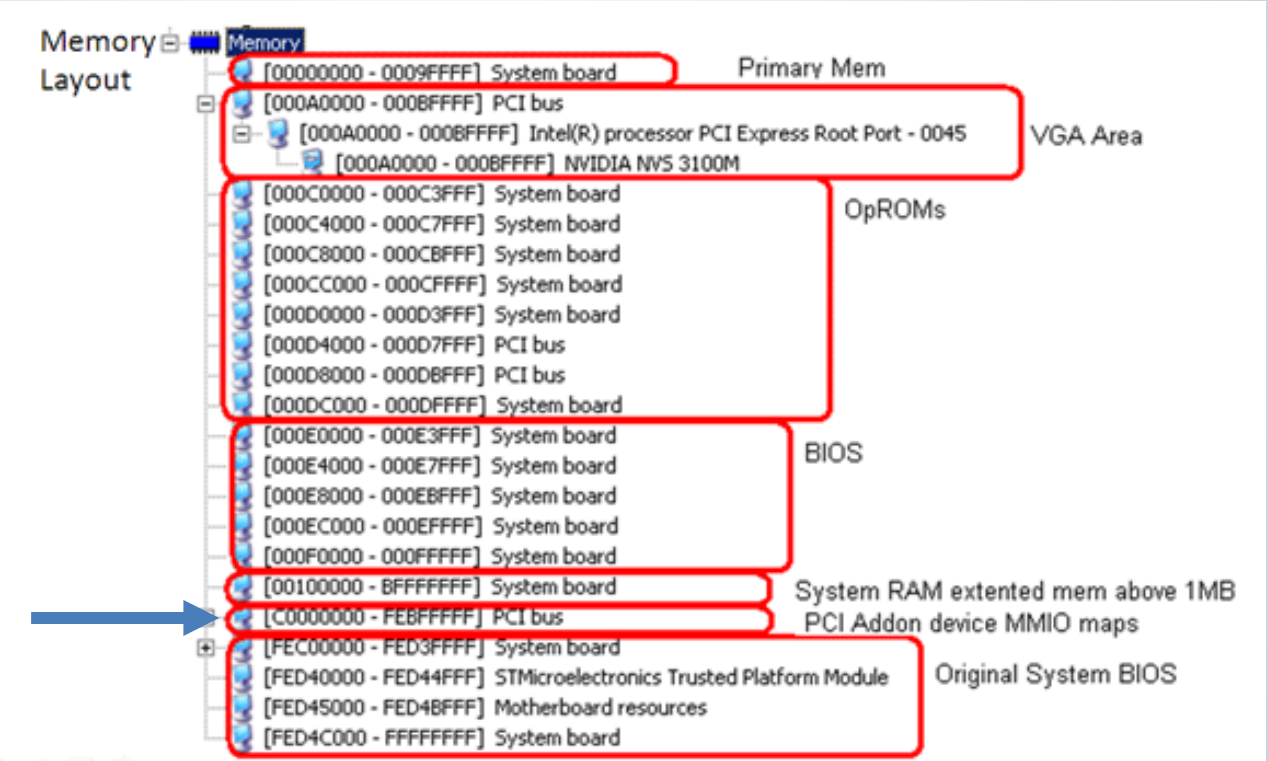
# Memory-mapped I/O (MMIO): treat devices like normal memory

- Certain physical addresses do not actually go to RAM
- Instead, they correspond to I/O devices
  - And any instruction that accesses memory can access them too!

- x86 being the historical amalgamation that it is, uses both PMIO or MMIO depending on the device



# Example memory map (from an old 32-bit computer)




PCI devices are mapped here

# Outline

- Overview of Device I/O
- Connecting to devices
  - Buses on a computer
- Talking to devices